COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/2/2026	**DATE SUBMITTED:**	10/2/2026
NAME:	Katindig, Jiro Luis	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 10.1 GRAPHS

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A Code

```
#include <iostream>
using namespace std;

struct adjNode {
    int val, cost;
    adjNode* next;
};

struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;

        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head;

        return newNode;
    }

    int N;

public:
    adjNode** head;
```

```
        DiaGraph(graphEdge edges[], int n, int N) {
            this->N = N;

            head = new adjNode*[N];

            for (int i = 0; i < N; ++i) {
                head[i] = nullptr;
            }

            for (int i = 0; i < n; ++i) {
                int start_ver = edges[i].start_ver;
                int end_ver = edges[i].end_ver;
                int weight = edges[i].weight;

                adjNode* newNode =
                    getAdjListNode(end_ver, weight, head[start_ver]);

                head[start_ver] = newNode;
            }
        }

        ~DiaGraph() {
            for (int i = 0; i < N; ++i) {
                adjNode* current = head[i];

                while (current != nullptr) {
                    adjNode* next = current->next;
                    delete current;
                    current = next;
                }
            }

            delete[] head;
        }
    };

    // A.1
    void displayVertices(int N) {
        cout << "Vertices: ";

        for (int i = 0; i < N; ++i) {
            cout << char('A' + i);
```

```
            if (i < N - 1)
                cout << ", ";
        }

        cout << endl;
    }



    // A.2
    void displayAdjMatrix(graphEdge edges[], int n, int N) {
        int matrix[10][10] = {};

        for (int i = 0; i < n; ++i) {
            int start = edges[i].start_ver;
            int end = edges[i].end_ver;
            int weight = edges[i].weight;

            matrix[start][end] = weight;
        }

        cout << "\nAdjacency Matrix\n\n";

        cout << "   ";
        for (int i = 0; i < N; ++i) {
            cout << char('A' + i) << " ";
        }
        cout << endl;

        for (int i = 0; i < N; ++i) {
            cout << char('A' + i) << " ";

            for (int j = 0; j < N; ++j) {
                cout << matrix[i][j] << " ";
            }

            cout << endl;
        }
    }

    // A.3
```

```cpp
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        cout << "(" << char('A' + i) << ", "
            << char('A' + ptr->val) << ", "
            << ptr->cost << ")" << endl;

        ptr = ptr->next;
    }
}

int main() {

    // Graph edges
    graphEdge edges[] = {
        {0, 1, 2},
        {0, 2, 4},
        {1, 4, 3},
        {2, 3, 2},
        {3, 1, 4},
        {4, 3, 3}
    };

    int N = 6;
    int n = sizeof(edges) / sizeof(edges[0]);

    DiaGraph diagraph(edges, n, N);

    cout << "A.1 Create a Graph" << endl;
    displayVertices(N);

    cout << "\nA.2 Adjacency Matrix" << endl;
    displayAdjMatrix(edges, n, N);

    cout << "\nA.3 Adjacency List" << endl;
    cout << "(Start Vertex, End Vertex, Weight)" << endl;

    for (int i = 0; i < N; ++i) {
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

ILO A Output

A.1 Create a Graph

```
A.1 Create a Graph
Vertices: A, B, C, D, E, F
```

A.2 Adjacency Matrix

```
A.2 Adjacency Matrix

Adjacency Matrix

  A B C D E F
A 0 2 4 0 0 0
B 0 0 0 0 3 0
C 0 0 0 2 0 0
D 0 4 0 0 0 0
E 0 0 0 3 0 0
F 0 0 0 0 0 0
```

A.3 Adjacency List

```
A.3 Adjacency List
(Start Vertex, End Vertex, Weight)
(A, C, 4)
(A, B, 2)
(B, E, 3)
(C, D, 2)
(D, B, 4)
(E, D, 3)
```

Analysis:

In this part, I made a directed graph with six vertices from A to F and six edges that each have a weight, then I stored it as an adjacency list, which means every vertex has its own list of the vertices it connects to, and the program showed the graph as a list of vertices, an adjacency matrix, and an adjacency list. The output showed that A connects to B with a weight of 2 and to C with a weight of 4, and the rest of the connections were the same as the edges I put in. The matrix is not the same on both sides, since A goes to B but B does not go back to A, so this shows that the graph is directed, and the row of F is all zeros because F has no connections. The adjacency list showed the same connections, but A to C came first before A to B, because the newest connection is placed at the front of the list.

ILO B Code

```cpp
#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <stack>
#include <queue>

using namespace std;

template <typename T>
struct Edge {
    size_t src;
    size_t dest;
    T weight;

    bool operator<(const Edge<T>& e) const {
        return this->weight < e.weight;
    }

    bool operator>(const Edge<T>& e) const {
        return this->weight > e.weight;
    }
};

template <typename T>
class Graph {
public:

    Graph(size_t N) : V(N) {}

    auto vertices() const {
        return V;
    }

    auto edges() const {
        return edge_list;
    }

    void add_edge(const Edge<T>& e) {
        if (e.src >= 1 && e.src <= V &&
```

```
            e.dest >= 1 && e.dest <= V) {
            edge_list.push_back(e);
        }
        else {
            cerr << "Vertex out of bounds" << endl;
        }
    }

    auto outgoing_edges(size_t v) const {
        vector<Edge<T>> edges_from_v;

        for (auto &e : edge_list) {
            if (e.src == v) {
                edges_from_v.push_back(e);
            }
        }

        return edges_from_v;
    }

private:
    size_t V;
    vector<Edge<T>> edge_list;
};

template <typename T>
ostream& operator<<(ostream& os, const Graph<T>& G) {

    for (auto i = 1; i < G.vertices(); i++) {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);

        for (auto &e : edges) {
            os << "(" << e.dest << ": " << e.weight << "), ";
        }

        os << endl;
    }

    return os;
};
```

```cpp
template <typename T>
auto create_reference_graph() {

    Graph<T> G(9);

    map<unsigned, vector<pair<size_t, T>>> edges;

    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {7, 0}, {4, 0}};
    edges[3] = {{4, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges) {
        for (auto &j : i.second) {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }

    return G;
}

// B.1
template <typename T>
auto depth_first_search(const Graph<T>& G, size_t dest) {

    stack<size_t> stack;
    vector<size_t> visit_order;
    set<size_t> visited;

    stack.push(1);

    while (!stack.empty()) {

        auto current_vertex = stack.top();
        stack.pop();

        if (visited.find(current_vertex) == visited.end()) {
```

```
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto &e : G.outgoing_edges(current_vertex)) {

                if (visited.find(e.dest) == visited.end()) {
                    stack.push(e.dest);
                }
            }
        }
    }

    return visit_order;
}

// B.2
template <typename T>
auto breadth_first_search(const Graph<T>& G, size_t dest) {

    queue<size_t> queue;
    vector<size_t> visit_order;
    set<size_t> visited;

    queue.push(1);

    while (!queue.empty()) {

        auto current_vertex = queue.front();
        queue.pop();

        if (visited.find(current_vertex) == visited.end()) {

            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto &e : G.outgoing_edges(current_vertex)) {

                if (visited.find(e.dest) == visited.end()) {
                    queue.push(e.dest);
                }
            }
```

```
        }
    }

    return visit_order;
}

// Test DFS
template <typename T>
void test_DFS() {

    auto G = create_reference_graph<unsigned>();

    cout << G << endl;

    cout << "DFS Order of vertices: " << endl;

    auto dfs_visit_order = depth_first_search(G, 1);

    for (auto v : dfs_visit_order) {
        cout << v << " ";
    }

    cout << endl;
}

// Test BFS
template <typename T>
void test_BFS() {

    auto G = create_reference_graph<unsigned>();

    cout << G << endl;

    cout << "BFS Order of vertices: " << endl;

    auto bfs_visit_order = breadth_first_search(G, 1);

    for (auto v : bfs_visit_order) {
        cout << v << " ";
    }

    cout << endl;
```

```
}

int main() {

    cout << "B.1 Depth-First Search" << endl;
    test_DFS<unsigned>();

    cout << "\nB.2 Breadth-First Search" << endl;
    test_BFS<unsigned>();

    return 0;
}
```

ILO B Output

B.1 Depth-First Search

```
B.1 Depth-First Search
1:      (2: 0), (5: 0),
2:      (1: 0), (7: 0), (4: 0),
3:      (4: 0),
4:      (2: 0), (3: 0), (5: 0), (6: 0), (8: 0),
5:      (1: 0), (2: 0), (4: 0), (8: 0),
6:      (4: 0), (7: 0),
7:      (3: 0), (6: 0),
8:      (4: 0), (5: 0), (6: 0),

DFS Order of vertices:
1 5 8 6 7 3 4 2
```

B.2 Breadth-First Search

```
B.2 Breadth-First Search
1:      (2: 0), (5: 0),
2:      (1: 0), (7: 0), (4: 0),
3:      (4: 0),
4:      (2: 0), (3: 0), (5: 0), (6: 0), (8: 0),
5:      (1: 0), (2: 0), (4: 0), (8: 0),
6:      (4: 0), (7: 0),
7:      (3: 0), (6: 0),
8:      (4: 0), (5: 0), (6: 0),

BFS Order of vertices:
1 2 5 7 4 8 3 6
```

Analysis:

In this part, I used a graph with 8 connected vertices to try Depth-First Search (DFS), and Breadth-First Search (BFS), and the program first showed the connections of each vertex, then it showed the order of how each search visited them. DFS gave the order 1 5 8 6 7 3 4 2, because it uses a stack, so it keeps going deeper into one path first and only goes back when it can't go any further. BFS gave the order 1 2 5 7 4 8 3 6, because it uses a queue, so it visits all the neighbors of a vertex first before moving on to the next level. Both searches visited each vertex only once, since the program keeps a record of the vertices that were already visited, and even if they start at the same vertex, the orders are different, which shows that the type of search changes the order of the visit.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

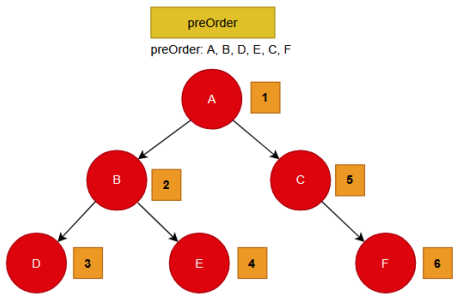
ILO C

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.
   - The algorithm that would be most helpful is the DFS (Depth First Search), because it goes deeper into one path first. When it reaches the end, it backtracks and explores another path. From the situation, this algo would be the best because the person wants to visit every spot on one location before moving on to other locations.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.
   - The equivalent of DFS in tree traversal is Pre-order Traversal. Both visit the current node first and then explore the connected nodes deeply before moving to another branch.

     For example:

     A → B → D → E → C → F



3. In the performed code, what data structure is used to implement the Breadth First Search?
   - A queue is used to implement Breadth First Search (BFS) because a queue follows the first-in, first-out rule, meaning the vertex that is added first will be processed first. This allows BFS to visit the graph level by level instead of going deeply into one path.

4. . How many times can a node be visited in the BFS
   - In the BFS, a node can only be visited once, then the program uses a visited set to keep track of the nodes that have already been visited. If the same node is encountered again, the program checks the set and skips it, preventing repeated visits and unnecessary processing.

Conclusion:

In this activity, I learned that a graph can be stored in different ways like an adjacency matrix or an adjacency list, and that it can be explored using DFS or BFS, and I also learned that the data structure used really matters, because a stack makes the search go deep first while a queue makes it go level by level. For the procedure, the outputs were the same as the edges I entered, and the results of DFS and BFS followed how each of them is supposed to work, so this showed me that the matrix is easier to read, while the list saves more memory when the graph only has a few connections.

For the supplementary activity, my answers showed that DFS is the better choice when a person wants to finish exploring one path before going back, and that the same idea in trees is called pre-order traversal, and I also answered that BFS uses a queue and that a node is only visited once because of the visited set, which matched the results I got in ILO B.

Overall, I think I did well in this activity because I was able to run the programs, get the right outputs, and understand the difference between DFS and BFS, but I still need to improve in understanding the code by myself, especially how the stack, queue, and linked lists work step by step, since I needed extra help for some parts, so I plan to practice by tracing the steps by hand and by writing more graph programs without depending too much on examples.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **More explanation of the topic**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I searched more about the topic, and asked for examples of codes in doing the coding part, since I didn't caught everything up when it was discussed**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.